EE-568 Applied Project : RLHF

Arnaud Teinturier **Diana Bejan** **Jiajun Hu** **Mathis Ayma**

Mengze Tian **Yule Chen** **Zhaoyu Zhu**

Group Feedback Fellows

https://github.com/mayma18/EE-568-RL

Abstract

In this project, we evaluate the performance of two Reinforcement Learning with Human Feedback (RLHF) algorithms, Direct Preference Optimization (DPO) and PPO-based RLHF, on three OpenAI Gym environments. Using preference data generated from expert and suboptimal policies, we analyze how the two algorithms perform across different preference dataset sizes and random seeds, providing a comparative view of their effectiveness and robustness. We find that PPO-based RLHF can outperform the expert when the preference dataset is large, while DPO shows instability when trained from scratch and benefits from regularization.

Introduction

RLHF offers a practical approach for aligning agent behavior with human preferences by learning from comparisons between trajectories, rather than relying solely on predefined reward functions. This project evaluates two RLHF algorithms, Direct Preference Optimization (DPO) and PPO-based RLHF, on CartPole, Pendulum and MountainCar environments.

We begin by training two policies using a PPO algorithm: π_1, attaining near-optimal rewards, and π_2, reaching a moderate reward with respect to π_1. Using these, we generate preference-labeled trajectory pairs. We then analyze how the two algorithms perform across different preference dataset sizes and random seeds, providing a comparative view of their effectiveness and robustness. The final version of the code is found in the link provided in the author region, under the branch **submit**.

1 Background

Reinforcement Learning from Human Feedback is a training framework where human judgments are used to guide reinforcement learning. Human feedback is collected in the form of preferences over outputs or trajectories, creating a dataset of comparisons between preferred and less preferred responses. This dataset is used to train a reward model that estimates how well an output aligns with human preferences.

Proximal Policy Optimization-based RLHF uses the reward model to guide the optimization of a new policy. A reference policy, usually obtained through supervised learning, is used to anchor the learning process. The new policy is trained to maximize reward according to the learned reward model while staying close to the reference policy. The degree of divergence is measured using the KL divergence and controlled by an alignment parameter β: higher values of β enforce stronger adherence to the reference policy. The optimization is performed using the PPO algorithm.

Direct Preference Optimization skips the reward model step and instead directly uses the preference dataset to adjust the model so that it assigns higher probability to preferred responses over rejected ones. This direct comparison-based objective encourages the model to generalize preferences, potentially favoring unseen actions that align with learned preferences. The use of the alignment parameter β is similar to that in PPO-based RLHF.

2 Approach

2.1 Preference Generation

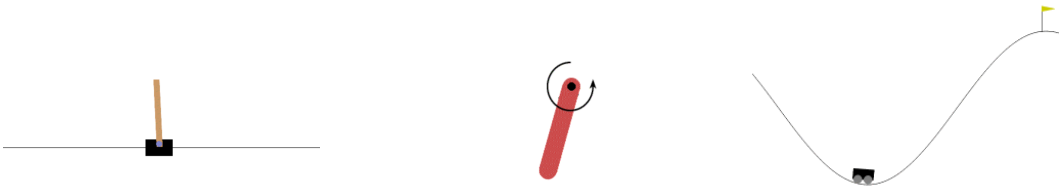
The goal of this section is to train the policies π_1 and π_2 as well as to generate a dataset of trajectory preferences that will be used in the two algorithms in the next section.

2.1.1 Environments Selection

First, we selected three OpenAI Gym environments, CartPole-v1, Pendulum-v1 and MountainCar-v0. These environments were chosen because they represent fundamental but distinct challenges in reinforcement learning: CartPole tests an agent's ability to maintain balance in a discrete action space, while Pendulum focuses on continuous control and energy-efficient planning. On the other hand, while MountainCar has a discrete action space, it is different from CartPole as reaching a goal is the termination and the length of the simulation represents penalties rather than rewards. The main parameters for each environment are seen in Table 1 and their visual representation in Figure 1.

Environment	States	Action Space	Rewards
CartPole	Cart position, cart velocity, pole angle, pole angular velocity	Discrete: push cart left (0) or right (1)	+1 for every time step the pole remains upright
Pendulum	Cosine of angle, sine of angle, angular velocity	Continuous torque in the range $[-2, 2]$	$-(\theta^2 + 0.1\,\dot{\theta}^2 + 0.001\,a^2)$, where θ is the angle, $\dot{\theta}$ is angular velocity, and a is the applied torque
MountainCar	Position, velocity	Discrete: push left (0), no push (1), push right (2)	-1 at each time step until the car reaches the goal position ($x \geq 0.5$)

Table 1: Comparison of three standard reinforcement learning environments



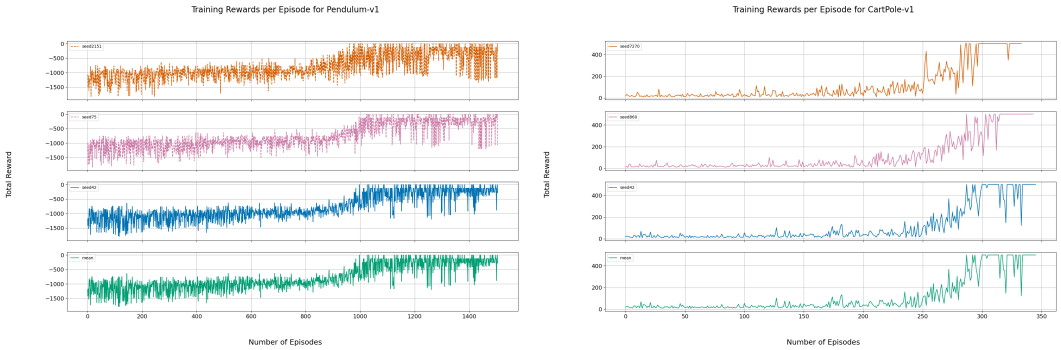
(a) CartPole environment (b) Pendulum environment (c) MountainCar environment

Figure 1: Visualization of the three reinforcement learning environments

2.1.2 Expert and Suboptimal Policies

The expert policy π_1 is trained using the PPO algorithm. The consistency of the training is verified by randomly selecting the seed of the environment for training, as seen in Figure 2. Since the initial training was deemed consistent enough in CartPole and Pendulum environments, no further

verification has been done on MountainCar. During training, checkpoints are saved and π_2 is selected from these. The choice falls on the checkpoint closest to a half-performance. For example, in the CartPole environment, the maximum reward is 500 while the minimum is 0. Since π_1 reaches the maximum reward, π_2 is chosen as the checkpoint with performance closest to a reward of 250.



(a) Pendulum (b) CartPole

Figure 2: Learning curves over 3 random seeds and the mean

2.1.3 Trajectory Generation

To generate a trajectory, we simply simulate the environment and log the state and action successions, as well as the reward earned at each step. Having noticed that the fine-tuning from the algorithms below is influenced by the length of the trajectories, we have decided to create trajectories of a fixed length each time. If a simulation is terminated before reaching the fixed length, it is padded with zeros. To account for the padding, the mask showing the valid length is also logged.

For both policies, K trajectories are generated, where K gives the size of the preference dataset. They are classified pairwise as preferred or rejected based on a softmax preference probability:

$$\frac{e^{R(\tau_1)}}{e^{R(\tau_1)} + e^{R(\tau_2)}}.$$

In the Cartpole environment, the action space is discrete, and the task is relatively straightforward. The small dataset consists of 200 trajectories, whereas the large dataset contains 1,000 trajectories. In contrast, the Pendulum environment features a continuous action space, making the task more complex and data-intensive than Cartpole. Its small dataset comprises 2,000 trajectories, while the large one includes 10,000. The MountainCar environment is notably sensitive to data variability. Here, the small dataset has just 10 trajectories, and the large dataset contains 100 trajectories.

2.2 RLHF Algorithms

2.2.1 PPO-based RLHF

Reward Models As noted above, we generated pairs of preferred and rejected trajectories using a near-optimal policy π_1 and a mediocre policy π_2. We treat π_1 as a proxy for a human expert and treat π_2 as the policy to be fine-tuned. Given these trajectory pairs, we train a neural network with the binary cross-entropy loss (nn.BCEWithLogitsLoss), forcing the network to score state–action pairs so as to distinguish between good and bad trajectories even without explicitly telling the network the reward values:

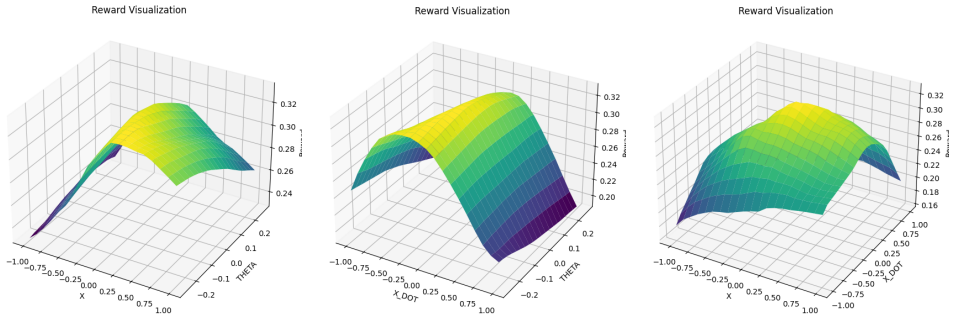
$$\mathcal{L}(\theta) \;=\; -\frac{1}{2}E\big[\log\sigma\big(r_\theta(y_w) - r_\theta(y_l)\big)\big] = \frac{1}{2}E\left[\log\left[\frac{e^{r_\theta(y_w)}}{e^{r_\theta(y_w)} + e^{r_\theta(y_l)}}\right]\right]$$

where y_w means the preferred trajectory and y_l means the rejected trajectory

PPO-RLHF policy After we trained the reward model, we replaced the reward function in the simulation environment with our reward model. We fine-tuned the policy π_2 with a restriction of the KL divergence between the policy and the expert KL $= 0.01$ and of the KL coefficient $\beta = 0.5$ to prevent the policy from changing too much in the new environment. In the CartPole environment, we trained the PPO-based RLHF with 20,000 steps, in the Pendulum environment with 200,000 steps, and in the MountainCar environment, 2,000 steps were used.

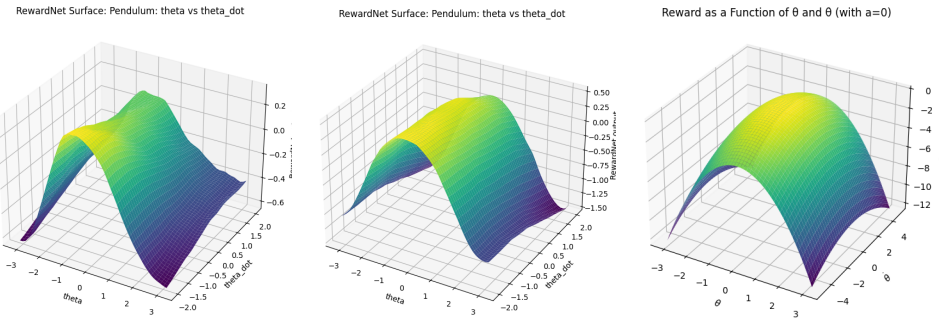
Results The PPO-RLHF fine-tuned from π_2 performs better than the original π_2 with 1000 trajectory pairs. In the CartPole simulation, we found that the PPO-RLHF approach even outperforms PPO policy π_1. The policy π_1 usually fails to stabilize the cart at the center position and shows a tendency to move towards the left side at the end of each episode. Therefore, even if both π_1 and PPO-RLHF (π_2) reach the maximum reward of 200, intuitively, PPO-RLHF (π_2) has a better approach to keep the pole in valid positions.

We attribute this difference to the environment's default reward model definition. The original reward function in CartPole assigns a reward of 1 as long as the cart and the pole stay within their respective bounds, i.e., the center of the cart is less than $\pm$ 2.4 and the pole angle is less than $\pm$ 12°. This reward is given regardless of the broader state dynamics. In contrast, our reward model assigns higher rewards to states reflecting better control and balance. For example, it assigns lower rewards when the cart's position (x) or velocity $(\dot{x})$ deviates from zero, thereby encouraging more stable and desirable behavior. The graphs in Figures 3 and 4 show the rewards the trained reward model gives across various states in two of the environments.



(a) Reward vs x and θ (b) Reward vs $\dot{x}$ and θ (c) Reward vs x and $\dot{x}$

Figure 3: Visualization of CartPole reward model under different states



(a) Reward vs x and θ 200 Pairs (b) Reward vs x and θ 10000 Pairs (c) Original Reward vs $\dot{x}$ and θ

Figure 4: Visualization of Pendulum reward model and original reward function

2.2.2 DPO

In the DPO framework, the policy is updated to increase the log-likelihood of preferred trajectories over less preferred ones. This is accomplished using a binary cross-entropy loss that encourages the model to assign higher probabilities to the preferred trajectories. Unlike PPO-RLHF, DPO does

not require a separate reward model, simplifying the training pipeline and reducing the potential for misalignment due to reward model inaccuracies.

We explored two DPO variants:

- **DPO from π_{ref}**: The policy is initialized from a suboptimal reference policy π_{ref} and fine-tuned using Direct Preference Optimization (DPO). In this variant, a KL-regularized objective is used to constrain the learned policy to remain close to the reference. This is motivated by the general RLHF objective used in PPO-based methods:

$$\max_{\pi_\theta} \quad \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta\, \mathbb{D}_{\text{KL}}\left[\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\right]$$

  where $r_\phi(x, y)$ is the reward given by a learned preference model, and β controls how strongly the policy stays close to π_{ref}.

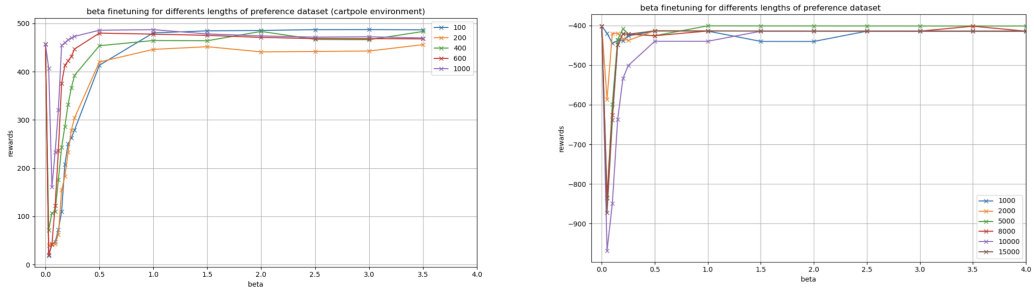
  The trajectory-level DPO loss then takes the form:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(\tau_1 \succ \tau_2)\sim\mathcal{D}}\left[\log\sigma\left(\beta\sum_{t=1}^{T}\left(\log\frac{\pi_\theta(a_1^t \mid s_1^t)}{\pi_{\text{ref}}(a_1^t \mid s_1^t)} - \log\frac{\pi_\theta(a_2^t \mid s_2^t)}{\pi_{\text{ref}}(a_2^t \mid s_2^t)}\right)\right)\right]$$

  where $\tau_1 \succ \tau_2$ denotes that trajectory τ_1 is preferred over τ_2, and $\sigma(\cdot)$ is the sigmoid function. The regularization coefficient β balances preference alignment and policy stability: high values enforce stronger adherence to π_{ref}, while low values allow more policy flexibility.

- **DPO from Scratch**: In this variant, the policy is initialized randomly and trained solely on the preference dataset without any reference policy. As a result, no KL regularization is applied, and the loss simplifies to:

$$\mathcal{L}_{\text{DPO-scratch}}(\pi_\theta) = -\mathbb{E}_{(\tau_1 \succ \tau_2)\sim\mathcal{D}}\left[\log\sigma\left(\sum_{t=1}^{T}\log\pi_\theta(a_1^t \mid s_1^t) - \sum_{t=1}^{T}\log\pi_\theta(a_2^t \mid s_2^t)\right)\right].$$



(a) CartPole environment

(b) Pendulum environment

Figure 5: Beta finetuning for different lengths of preference dataset

Effect of β and preference dataset size on DPO: The performance of DPO is jointly influenced by the KL coefficient β and the size of the preference dataset, with different dynamics observed across environments. In the CartPole environment (Fig. 5a), small β values (e.g., $\beta < 0.2$) consistently lead to lower rewards, especially with smaller datasets. This is because a weak regularization allows the policy to drift too far from the reference π_2, without sufficient preference signal to guide it effectively. As the dataset size increases, this trend changes: the stronger preference signal enables the policy to benefit from smaller β. For example, when the dataset size exceeds 600, the highest rewards are achieved with $\beta = 0.5$, while smaller datasets still require $\beta \approx 1$ to reach similar performance.

This indicates that with richer preference data, the policy can safely deviate more from π_2, but some regularization remains necessary to achieve optimal performance.

In contrast, the Pendulum environment (Fig. 5b) presents a more challenging learning scenario. At small β values (especially $\beta < 0.2$), larger preference datasets actually result in lower rewards. This suggests that a stronger preference signal pushes the policy further from π_2, but without leading to better performance — likely due to the higher complexity of the environment and the limited generalization capability of the policy. As a result, rewards drop below the baseline provided by π_2. Only when β is increased (typically above 0.5) does the performance stabilize and return to levels close to or slightly above π_2. This indicates that in complex tasks like Pendulum, strong regularization is critical to prevent the model from overfitting to misleading preference signals and to ensure stable learning. Overall, these results highlight the need to jointly tune β and dataset size depending on the environment's complexity and the quality of preference data.

3 Analysis

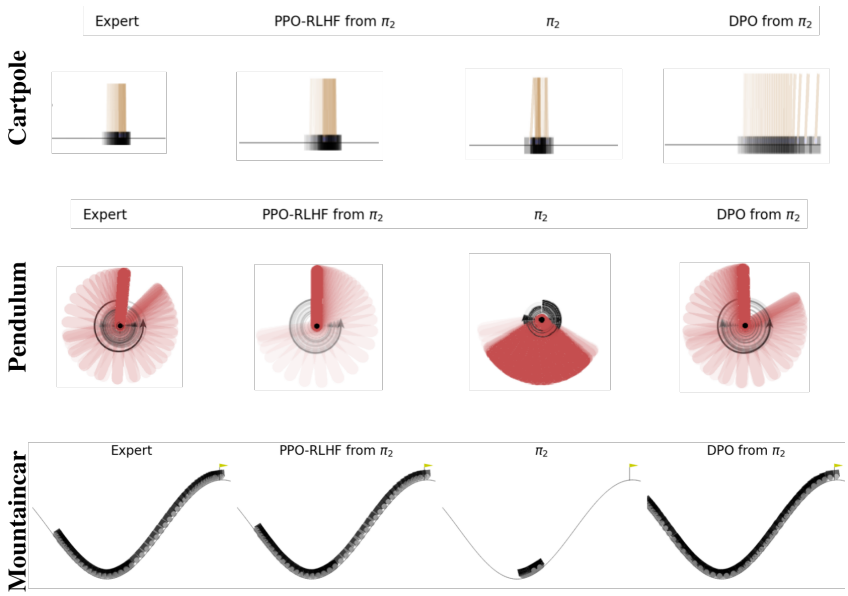


Figure 6: Performance comparison of Expert, PPO-RLHF from π_2, π_2 and DPO from π_2 trained in large dataset. The agent's after-images illustrate its motion over time.

The comparison of different preference dataset sizes is shown for each environment in Figure 7. To allow for generalized conclusions, 20 complete flows have been run and compiled in each case. Generally speaking, we find that PPO-based RLHF outperforms DPO when the preference dataset is large. In such cases, PPO-based RLHF can even surpass the performance of the expert model, suggesting that learning a reward function with sufficient preference data can lead to effective generalization. However, in the sensitive MountainCar task, when the preference dataset is small, PPO-based RLHF is comparable to DPO, with both methods showing limited effectiveness. We believe this is due to overfitting in the reward model caused by the limited and noisy preference data, making it difficult to extract useful training signals.

For DPO, we observe only a slight improvement over the reference policy π_2 in the Pendulum task, even with a larger dataset. We attribute this to the task's continuous action space, which makes it harder for DPO to generalize across different state-action distributions. Our findings suggest that DPO relies more heavily on high-quality data to perform well and is generally less robust than PPO-based RLHF in this setting.

Interestingly, while DPO struggles in the Pendulum task, it performs much better in the MountainCar environment. We believe this is due to the structure of the problem, which requires the car to first move left before accelerating to the right. In this task, the policies π_1 and π_2 differ more substantially,

and the preference data is both higher in quality and more representative. As a result, DPO benefits from clearer learning signals and achieves good performance in this setting.

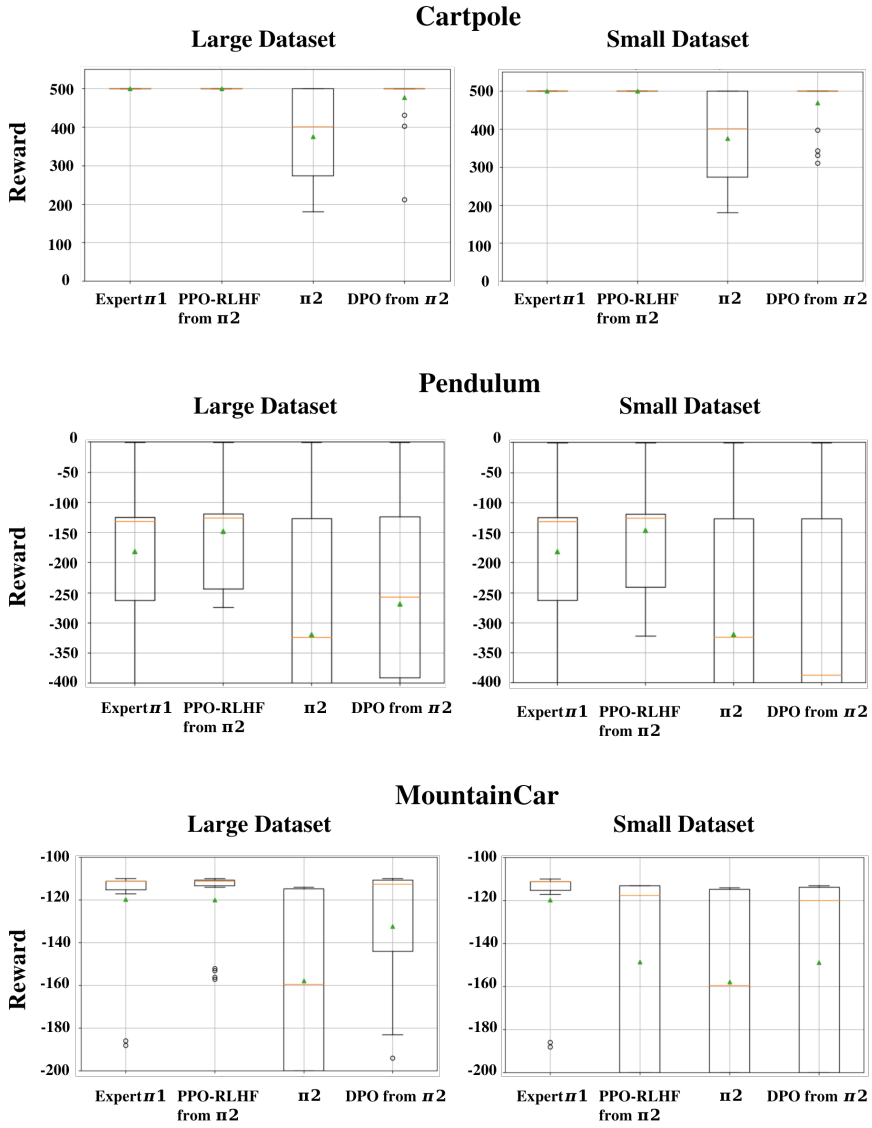


Figure 7: Results on the chosen environments with different preference dataset sizes

In Figure 6, the visual representation of the policies' tendencies can be seen. A darker color indicates that the object in movement has been in that specific place more times than elsewhere. In the CartPole after-images, we notice that the expert and the PPO-RLHF policy keep the pole more stable and upward pointing than π_2 and the DPO policy. In the Pendulum after-images, both DPO and PPO-RLHF achieve substantial improvements over π_2, progressing from an inability to swing up to successfully maintaining an upright position. Notably, PPO-RLHF reaches the upright state more rapidly and exhibits the smallest swing amplitude before stabilization, suggesting that the reward function learned from preference data captures the environment dynamics more accurately than the original reward provided by Open Gym. In the MountainCar traces, π_2 fails to reach the summit and performs the worst; both PPO-RLHF and DPO deliver significant gains over π_2. PPO-RLHF matches the performance of the expert policy π_1, while DPO falls slightly short of π_1.

4 Conclusion

Our findings indicate that preference dataset size significantly affects the performance of RLHF methods, especially in complex environments like Pendulum and MountainCar. While PPO-based RLHF can outperform expert policies with sufficient data, DPO shows instability when trained from scratch and benefits from regularization. For simpler environments, smaller datasets suffice, but in complex domains, careful tuning of the KL coefficient β and preference dataset size is crucial. Overall, the complexity of PPO-based RLHF is justified when working with complex environments.

References

[1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.

[2] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[3] Raphael Rafailov, Yiding Ge, Micah Goldblum, Tom Goldstein, Honglak Zhang, Pavel Izmailov, and Andrew Gordon Wilson. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.